下面给你一份**SCA** 系统验证 **checklist**。目标不是“能跑”，而是确认：

- 数据没错
- 负采样没错
- forward 真按 SCA 逻辑执行
- `delta_u / gate / control injection` 真在起作用
- 梯度和参数更新正常
- 训练行为符合预期
- 关掉 semantic control 后，系统表现真的不同

我按你要的顺序来。

---

0. 总原则

每次只验证一层，不要一口气看最终 loss。

正确顺序是：

1. 数据层
2. batch sampling
3. forward 中间变量
4. loss 数值
5. gradient
6. parameter update
7. training behavior
8. 最小 ablation

每一层通过后再看下一层。

---

1. 数据层检查

1.1 数据集基本规模

打印

- `num_users`

- `num_items`
- `len(train_pairs)`
- `len(valid_pairs)`
- `len(test_pairs)`

正常现象

- 都大于 0
- `num_users`、`num_items` 和你的处理后数据规模一致
- `train_pairs` 明显多于 0，且通常大于 valid/test

有 bug 的信号

- 某个 split 长度是 0
- `num_users` 或 `num_items` 明显偏小
- train/valid/test 数量离谱，比如 train 很少但 test 很多

1.2 user-item matrix 和 graph

打印

- `user_item_matrix.shape`
- `norm_adj.shape`
- 稀疏矩阵非零元数量（如果你有 `_nnz()` 之类接口）

正常范围

- `user_item_matrix.shape == (num_users, num_items)`
- `norm_adj.shape == (num_users + num_items, num_users + num_items)`
- `user_item_matrix` 的非零元数量约等于 `len(train_pairs)`
- `norm_adj` 的非零边数量约等于 `2 * len(train_pairs)`（双向边）

有 bug 的信号

- shape 对不上
- `user_item_matrix` 非零元远小于 train interactions
- `norm_adj` 稀疏边数明显不合理
- 某些用户在 train 里有交互，但在 `user_item_matrix` 聚合时表现成全零

---

2. Batch sampling 检查

2.1 batch 基本 shape 和范围

打印

取一个 batch，打印：

- `user_ids[:10]`
- `pos_item_ids[:10]`
- `neg_item_ids[:10]`
- 三者的 shape

正常范围

- shape 都应为 `(B,)`
- `0 <= user_id < num_users`
- `0 <= item_id < num_items`

有 bug 的信号

- shape 不是一维
- 出现负数 id 或越界 id
- dtype 不是整型 tensor

2.2 负采样正确性

打印

随机检查 batch 中前几个样本：

- `(u, pos, neg)`
- 以及 `neg in train_user_pos_dict[u]`

正常范围

- 对每个样本，neg 都不应该在该用户的正样本集合里

有 bug 的信号

- 只要出现一次 neg 实际是正样本，就说明 BPR 负采样有错

2.3 batch 覆盖情况

打印

- 一个 epoch 里 unique user 数量
- 一个 epoch 里 unique pos item 数量

正常范围

- 不会极端集中到很少一部分用户/物品
- shuffle 后每个 epoch 分布应该比较自然

有 bug 的信号

- batch 总是重复同一小撮 user
- user 或 item 分布异常集中，可能是 dataset / dataloader 出错

---

3. Forward 检查

这一部分最关键。先只跑一个 **batch forward**，不 **backward**。

3.1 collaborative embeddings

打印

- `user_all_embeddings.shape`
- `item_all_embeddings.shape`
- `e_u.shape`
- `pos_item_emb.shape`

- `neg_item_emb.shape`

正常范围

若 embedding 维度为 `D`:

- `user_all_embeddings == (num_users, D)`
- `item_all_embeddings == (num_items, D)`
- `e_u == (B, D)`
- `pos_item_emb == (B, D)`
- `neg_item_emb == (B, D)`

有 bug 的信号

- shape 不对
- `e_u` 和 item embedding 维度不一致
- batch index 后 shape 异常

3.2 semantic branch：`z_u` 和 `delta_u`

打印

- `z_u.shape`
- `delta_u.shape`
- `z_u.mean()`
- `z_u.std()`
- `delta_u.mean()`
- `delta_u.std()`
- `delta_u.abs().mean()`

正常范围

- `z_u.shape == (B, semantic_dim)`
- `delta_u.shape == (B, D)`
- `std` 不应接近 0
- `delta_u.abs().mean()` 应明显大于 0

有 bug 的信号

- `delta_u` 全零或几乎全零
- `delta_u.std()` 非常接近 0
- `z_u` 对所有用户几乎一样，说明 semantic encoder 退化了

3.3 structural context：`c_u`

打印

- `c_u.shape`
- `c_u.mean()`
- `c_u.std()`
- `c_u.abs().mean()`

正常范围

- `c_u.shape == (B, D)`
- `c_u.abs().mean()` 应大于 0
- `c_u.std()` 不应接近 0

有 **bug** 的信号

- `c_u` 大量全零
- `c_u` 对不同 user 几乎一样
- shape 不对，通常说明邻域聚合实现有问题

3.4 gate：`g_u`

打印

- `g_u.shape`
- `g_u.min()`
- `g_u.max()`
- `g_u.mean()`
- `g_u.std()`

正常范围

- 如果是 vector gate：`g_u.shape == (B, D)`
- 所有值应在 `[0, 1]`
- `mean` 不应长期贴近 0 或 1
- `std` 应大于一个很小的量，说明不是常数门

有 bug 的信号

- `g_u` 全接近 0：semantic control 被完全关掉
- `g_u` 全接近 1：gate 失去调节作用
- `g_u.std()` 极小：所有用户几乎同一个 gate
- 值跑出 `[0,1]`：激活或实现有问题

3.5 semantic control 是否真的改了 user embedding

打印

- `e_u.abs().mean()`
- `user_emb.abs().mean()`（控制后的）
- `(user_emb - e_u).abs().mean()`

正常范围

- `(user_emb - e_u).abs().mean()` 应明显大于 0
- 但不应大到比 `e_u.abs().mean()` 还夸张很多

有 bug 的信号

- 差值几乎为 0：semantic control 没有真正生效
- 差值特别大：`delta_u` 或 gate 可能失控

---

4. Loss 检查

4.1 loss 组成

打印

每个 batch 或每个 epoch：

- `loss`
- `bpr_loss`
- `align_loss`
- `reg_loss`

正常范围

- 都应是有限值
- 都不应是 `nan` / `inf`
- 一般 `loss > 0`

有 bug 的信号

- 任何一个是 `nan`
- `align_loss < 0`（如果你的定义不允许）
- `loss` 爆炸到非常大且不收敛

4.2 `pos_scores` vs `neg_scores`

打印

- `pos_scores.mean()`
- `neg_scores.mean()`
- `(pos_scores > neg_scores).float().mean()`

正常范围

训练初期：

- 不一定差很多，但至少不应长期完全反过来

训练几轮后：

- `pos_scores.mean()` 应高于 `neg_scores.mean()`
- `pos > neg` 的比例应逐渐大于 0.5，并继续提升

有 bug 的信号

- 长期 `pos_scores.mean() <= neg_scores.mean()`
- `pos > neg` 比例长期在 0.5 附近不动
- 或者一直低于 0.5

4.3 alignment loss 的合理性

打印

- `align_loss`
- `cos(delta_u, c_u)` 的均值（如果你方便看）

正常范围

- alignment loss 应该大于 0
- 随训练通常会下降或变稳
- 如果是 cosine 对齐，相似度应有上升趋势

有 bug 的信号

- alignment loss 长期不变
- `delta_u` 和 `c_u` 完全无关
- 开关 alignment 后结果完全一样

---

5. Gradient 检查

第一次 backward 时一定要查。

5.1 关键模块梯度

打印

这些参数的 gradient norm：

- backbone user embedding
- backbone item embedding
- semantic encoder 主层
- projector

- gate 层

正常范围

- 梯度不是 `None`
- 梯度范数大于 0
- 梯度不是 `nan`

有 bug 的信号

- 某模块梯度是 `None`
- 某模块梯度长期为 0
- gate / projector 没梯度，通常说明 semantic 路径没接进 loss
- backbone 没梯度，说明 ranking 路径断了

---

6. Parameter update 检查

只做一次 step 前后比较。

6.1 参数是否真的更新

打印

保存 step 前后的参数差异，例如：

- projector 权重平均绝对变化
- gate 权重平均绝对变化
- backbone embedding 平均绝对变化

正常范围

- 都应大于 0

有 bug 的信号

- backward 有梯度，但 step 后参数没变
- 某些关键模块始终不更新

---

7. Training behavior 检查

这个阶段不是看单步，而是看趋势。

7.1 每个 epoch 建议打印的核心统计

最值得长期打印的是：

- `loss`
- `bpr_loss`
- `align_loss`
- `pos_scores.mean()`
- `neg_scores.mean()`
- `(pos_scores > neg_scores).float().mean()`
- `delta_u.abs().mean()`
- `g_u.mean()`
- `g_u.std()`
- `(user_emb - e_u).abs().mean()`

7.2 正常训练趋势

正常现象

- `loss` 逐渐下降
- `bpr_loss` 逐渐下降
- `align_loss` 下降或变稳
- `pos_scores.mean() - neg_scores.mean()` 变大
- `pos > neg` 比例上升
- `delta_u.abs().mean()` 不为 0
- `g_u.mean()` 在合理区间，不贴边
- `g_u.std()` 不为 0
- `(user_emb - e_u).abs().mean()` 大于 0 且比较稳定

有 bug 的信号

- loss 完全不动
- `pos` 和 `neg` 分数长期没差

- gate 坍缩
- semantic control 开着，但 `(user_emb - e_u)` 基本为 0
- `delta_u` 慢慢塌成 0

---

8. 你特别关心的四件事：单独检查

8.1 `delta_u` 是否真的非零

看什么

- `delta_u.abs().mean()`
- `delta_u.std()`

正常

- 两者都明显大于 0

异常

- 接近 0：projector 或 semantic encoder 形同虚设

8.2 gate 是否退化

看什么

- `g_u.mean()`
- `g_u.std()`
- `g_u.min()`, `g_u.max()`

正常

- 有分布，不贴边，不是常数

异常

- 全接近 0
- 全接近 1
- std 极小

8.3 semantic control 是否真的改变 user embedding

看什么

- `(user_emb - e_u).abs().mean()`

正常

- 明显大于 0

异常

- 接近 0，说明控制支路没生效

8.4 `pos_scores vs neg_scores` 是否合理

看什么

- `pos_scores.mean()`
- `neg_scores.mean()`
- `pos > neg` 比例

正常

- 训练后 `pos > neg` 应逐渐占优

异常

- 长期差不多或反过来

---

9. 最小 ablation：验证 SCA 真的产生不同结果

这个必须做，而且很容易做。

9.1 如何关闭 semantic control

最小方法有两种，任选一种：

方法 A：设 `control_scale = 0`

这样：

e~u=eu+0⋅(gu⊙Δu)=eu\tilde e_u = e_u + 0 \cdot (g_u \odot \Delta_u) = e_ue~u=eu+0⋅(gu⊙Δu)=eu

方法 B：强制 `delta_u = 0`

也能达到同样效果。

方法 C：强制 `g_u = 0`

效果也一样。

最干净的是 **A：control_scale = 0**，因为不改模型结构，只关掉控制注入。

9.2 关掉后应该比较什么

对比两种设置：

1. **Full SCA**
2. **Semantic control off**

至少比较以下量：

- `loss`
- `bpr_loss`
- `align_loss`
- `pos_scores.mean()`

- `neg_scores.mean()`
- `pos > neg` 比例
- `(user_emb - e_u).abs().mean()`

9.3 预期现象

Full SCA

- `(user_emb - e_u).abs().mean()` 明显大于 0
- `delta_u`、`g_u` 有实际作用
- loss / ranking 行为应与 baseline 有差异

Control off

- `(user_emb - e_u).abs().mean()` 应接近 0
- 打分只来自协同表示
- 与 Full SCA 的训练曲线或中间统计应不同

9.4 什么现象说明你的 SCA 其实没起作用

如果 Full SCA 和 control-off 的结果几乎完全一样，包括：

- `loss`
- `bpr_loss`
- `pos/neg score`
- `pos > neg` 比例
- 训练曲线

都几乎没区别，那么基本说明：

- semantic 分支虽然在算，但没进入最终决策
- 或者 gate / delta 被压没了
- 或者 control injection 实现无效

---

10. 最后给你一个最实用的执行顺序

你照这个顺序做就行：

Step A：静态检查

看数据规模、矩阵 shape、稀疏边数

Step B：batch 检查

看 user / pos / neg 范围，检查 neg 不是正样本

Step C：单 batch forward

看：

- `delta_u`
- `c_u`
- `g_u`
- `(user_emb - e_u).abs().mean()`
- `pos_scores / neg_scores`

Step D：单 batch backward

看关键模块 gradient norm

Step E：单步更新

看关键参数前后是否变化

Step F：短训练

跑 3 到 5 个 epoch，看训练趋势

Step G：最小 ablation

把 `control_scale=0`，和 Full SCA 对比中间统计与训练曲线

---

11. 一组最值得长期监控的核心指标

如果你只想保留最关键的一组，我建议每个 epoch 固定看这 10 个：

- `loss`
- `bpr_loss`
- `align_loss`
- `pos_scores.mean()`
- `neg_scores.mean()`
- `(pos_scores > neg_scores).float().mean()`
- `delta_u.abs().mean()`
- `g_u.mean()`
- `g_u.std()`
- `(user_emb - e_u).abs().mean()`

这 10 个基本就能判断你的 SCA 是不是“真的在工作”。